MODULE 02 · DOMAIN 1 - MONITORING, LOGGING, ANALYSIS, REMEDIATION, AND PERFORMANCE OPTIMIZATION

Alarms That Work

States, evaluation windows, missing data, composite alarms and mute rules. Most broken alarms are broken in the same four ways.

DOMAIN 1 of 5 EXAM WEIGHT 22% MODULE 2 of 35

admin@sthithapragna.com

An alarm is not a threshold check. It is a **state machine voting over a window**, and the window is where the surprises live.

Three states, and what each one means

- **OK.** The metric is inside the threshold for the evaluated window.
- **ALARM.** The metric is outside the threshold for enough of the window.
- **INSUFFICIENT_DATA.** Not enough data points to decide. Not a failure, and not a success.
- You can set actions on **all three transitions**, including recovery back to OK.

> ON THE EXAM
>
> A new alarm always starts in INSUFFICIENT_DATA. So does an alarm whose metric has gone quiet, which are two very different situations.

How an alarm actually decides

01

Period

How long one data point covers. 60 seconds, 300 seconds, and so on.

02

Evaluation periods

How many of those periods CloudWatch looks at. The N.

03

Datapoints to alarm

How many of those N must breach. The M.

04

Missing data rule

What to do about periods with no data at all.

05

State

OK, ALARM or INSUFFICIENT_DATA, then actions on transition.

ON THE EXAM

M out of N is the whole design. 3 of 3 is slow and steady; 2 of 5 tolerates a flapping metric without ignoring it.

Why an alarm can be OK while the latest data point is breaching

- The alarm needs **M breaching points out of N**. One bad point out of three changes nothing, by design.
- To fill gaps, CloudWatch looks back over an **evaluation range wider than N**, pulling in older points.
- So a fresh breach can be **outvoted by older healthy data** that is still inside the range.
- Want it to react to the newest point? Use **1 datapoint out of 1** and accept the noise that comes with it.

> ON THE EXAM
>
> "The graph is clearly over the line but the alarm says OK" is almost always M out of N, not a bug.

The four missing-data treatments

SETTING	WHAT CLOUDWATCH DOES WITH AN EMPTY PERIOD
`missing (the default)`	Judge on the data points that do exist. If none exist at all, go to INSUFFICIENT_DATA
`notBreaching`	Treat it as good. Pushes the alarm toward OK
`breaching`	Treat it as bad. Pushes the alarm toward ALARM
`ignore`	Do not re-evaluate. The alarm holds whatever state it already had

ON THE EXAM

Alarms on metrics in the AWS/DynamoDB namespace default to ignore, not missing. That exception is examinable.

Treat missing as breaching, or not breaching?

NOTBREACHING WHEN

Silence is normal

- The metric only publishes when something happens
- Error counts, throttles, queue depth on a quiet queue
- You want no news to mean good news

BREACHING WHEN

Silence is the failure

- A heartbeat metric published by a job that must run
- An agent that should always be reporting
- The absence of data is the symptom you care about

THE TIEBREAKER

Ask what silence means for this metric. If a stopped process stops publishing, silence is the outage and breaching is correct.

What an alarm can do when it fires

ACTION	NOTES THAT DECIDE QUESTIONS
Notify an SNS topic	The default. Fan out to email, chat, a pager, a queue, or Lambda
Invoke a Lambda function	Supported directly, no SNS topic in the middle
EC2 action	Stop, terminate, reboot or recover. Only on alarms watching EC2 metrics
Auto Scaling action	Runs a step or simple scaling policy on the group
Systems Manager	Create an OpsItem in OpsCenter, the recommended path. ALARM state only
EventBridge event	Every state change emits an event. This is the general-purpose hook

ON THE EXAM

Except for Auto Scaling actions, actions fire on the TRANSITION only. An alarm stuck in ALARM for two days notifies once.

EC2 status checks and the recover action

- **StatusCheckFailed_System** means the underlying host or its network is broken. **Recover** is the right response.
- **StatusCheckFailed_Instance** means the guest is broken. Reboot or replace it. Recovery will not help.
- Recover migrates to new hardware and **keeps the instance ID, private IP, Elastic IP and metadata**.
- **Instance store data is lost.** The instance is restarted on different hardware, not moved.

> ON THE EXAM
>
> System check means AWS's problem, recover. Instance check means your problem, reboot. Picking the wrong one is a favourite distractor.

Static threshold or anomaly detection?

STATIC THRESHOLD WHEN

You know the number

- An SLA, a quota, a disk that must not fill
- Behaviour is flat, or you genuinely have a limit
- Cheap, obvious, easy to explain in a review

ANOMALY DETECTION WHEN

Normal is a shape, not a number

- Traffic with daily and weekly cycles
- A fixed threshold would be wrong at 3am or wrong at noon
- Builds a band from up to two weeks of history

THE TIEBREAKER

If the scenario says the threshold is right on weekdays and wrong at weekends, it is describing seasonality. That is an anomaly detection band.

Composite alarms: one incident, one page

- A composite alarm has **no metric**. Its state comes from a rule over other alarms.
- Rule syntax uses **ALARM(name)**, **OK(name)**, **INSUFFICIENT_DATA(name)** with AND, OR and NOT.
- Put notifications on the composite and **remove them from the children**. Twenty alarms, one page.
- Composites can notify SNS and raise OpsItems or incidents. They **cannot do EC2 or Auto Scaling actions**.

> ON THE EXAM
>
> "One failure generates forty emails" is the composite alarm signal. Silence the children, alert on the parent.

Suppressing noise: two different tools

ACTIONS SUPPRESSOR

Driven by another alarm's state

- Set on a composite alarm
- While the suppressor is in ALARM, actions are held
- WaitPeriod and ExtensionPeriod cover lag
- For: network down, do not page about the app

ALARM MUTE RULES

Driven by the calendar

- Announced February 2026. One-time or recurring
- cron or at expressions, plus a duration and timezone
- Up to 100 alarms per rule
- For deployments, maintenance windows, off-hours

THE TIEBREAKER

Is the reason for the silence a condition, or a schedule? Condition means suppressor. Schedule means mute rule.

Mute rules: three details worth knowing

- **Alarms keep evaluating** and keep changing state. Only the actions are muted, so history stays intact.
- When a rule expires, **previously muted actions fire** if the alarm is still in the same state.
- Replaces the old workaround: a scheduled Lambda disabling and re-enabling alarm actions.
- Creating one needs **PutAlarmMuteRule on the rule and on every targeted alarm**.

> ON THE EXAM
>
> The old pattern left alarms disabled when the re-enable script failed. Mute rules expire on their own, which is the real argument for them.

Alarms on things that are not a plain metric

- **Metric math.** Alarm on an error percentage rather than on a raw error count. Far fewer false alarms at low traffic.
- **High-resolution alarms.** Periods of 10 or 30 seconds, only on high-resolution custom metrics.
- **Search expressions** must be collapsed with metric math first, because an alarm needs a single series.
- Cross-account alarms work, but **not ANOMALY_DETECTION_BAND, INSIGHT_RULE or SERVICE_QUOTA**, and not cross-account composites.

> ON THE EXAM
>
> Ten errors out of ten requests is an outage. Ten out of a million is Tuesday. Alarm on the ratio, not the count.

Six things an alarm can watch

DATA SOURCE	WHEN IT IS THE RIGHT CHOICE
A single metric	The default. One resource, one number
A metric math expression	Ratios, sums across a fleet, anomaly detection bands
A Metrics Insights query	A whole fleet at once, adjusting as resources come and go
A log group metric filter	A pattern in logs that you also want to graph
A Logs Insights query (log alarm)	A pattern in logs, with NO metric filter to maintain
A PromQL instant query	OpenTelemetry metrics sent to the CloudWatch OTLP endpoint

ON THE EXAM

A log alarm removes a whole step. You no longer need a metric filter just to alarm on something in a log.

One alarm for a whole fleet

- Alarm on a Metrics Insights query with **GROUP BY**, and each returned series becomes a **contributor**.
- The query re-runs at every evaluation, so the alarm **adapts as resources appear and disappear**. No alarm to create, none to clean up.
- Notifications name the breaching contributor, and **EC2 actions run at contributor level**, on the instance that actually breached.
- Limits: **100 contributors in ALARM**, only the last **3 hours** of data, 200 such alarms per account per Region by default.

> ON THE EXAM
>
> "Alarms must cover instances that do not exist yet, without a script" is a Metrics Insights alarm. A per-instance alarm cannot do it.

The INSUFFICIENT_DATA checklist

- **Is the period shorter than the publishing interval?** A 60-second period on basic monitoring data is the classic.
- **Did the dimension set change?** The alarm watches an exact coordinate. Module one covered this.
- **Does the metric only publish when something happens?** Then this is normal, and missing-data treatment is the fix.
- **Is the resource gone?** A terminated instance stops publishing, and its alarms sit there for ever.

> ON THE EXAM
>
> Work down the list in order. Three of the four are configuration, and only the last is a real incident.

> A nightly batch job publishes a custom metric called JobHeartbeat, with a value of 1, once at the end of each successful run. An alarm watches it with a period of one hour, one evaluation period, and a threshold of less than 1.
>
> The job failed to start for three consecutive nights. Nobody was notified. The alarm has been in INSUFFICIENT_DATA throughout, and the operations lead wants to know why it did not fire.

What change makes this alarm detect the failure?

A Set the missing data treatment to breaching

B Set the missing data treatment to notBreaching

C Increase evaluation periods to 3 so three nights are considered

D Add an INSUFFICIENT_DATA action pointing at the SNS topic

◆ PAUSE AND DECIDE

What change makes this alarm detect the failure?

A Missing data as breaching

Silence is the failure here. A job that does not run publishes nothing, so an empty period must count as bad.

B Missing data as notBreaching

Exactly backwards. It would guarantee the alarm never fires, because the failure mode produces no data at all.

C Three evaluation periods

Changes how many periods vote. It does not change the fact that every one of those periods is empty and cannot vote.

D INSUFFICIENT_DATA action

The genuinely tempting one. It would notify, but it also notifies for every hour of the twenty-three when no run is expected.

THE SIGNAL

The metric only exists on success, so absence is the signal. That is the definition of the breaching treatment.

> An RDS database instance fails. Within two minutes, 26 CloudWatch alarms move into ALARM: connection counts on six application services, latency on four load balancers, queue depth on three queues, and error rates across the rest.
>
> Every alarm notifies the same SNS topic, which pages the on-call engineer. The team wants one page per incident, while keeping every alarm's history for post-incident review.

What should the team implement?

A. Delete the child alarms and keep only the database alarm

B. Create a composite alarm over the children, move the SNS action to it, and remove the actions from the children

C. Create an alarm mute rule covering the 26 alarms

D. Raise the datapoints to alarm on every child so they take longer to fire

◆ PAUSE AND DECIDE

What should the team implement?

A Delete the child alarms

Solves the paging by destroying the evidence. The requirement explicitly asks to keep the history for review.

B Composite alarm, actions moved to the parent

Children keep evaluating and keep their history; only the composite pages. One incident, one notification.

C Alarm mute rule

Mute rules are scheduled. A database failure is not on the calendar, so there is no window to define.

D Raise datapoints to alarm

Delays twenty six pages rather than preventing them. It also slows genuine detection, making the system worse.

THE SIGNAL

Two requirements: one page, and keep the history. Only the composite pattern satisfies both, because muting actions is not the same as deleting alarms.

A retail platform alarms on request latency. The threshold is a static 400 milliseconds, with a five-minute period and three datapoints out of three. It pages reliably during the weekday morning peak, when latency naturally rises, and the team has learned to dismiss those pages.

During a quiet Sunday night, latency doubled from its usual 80 milliseconds to 160 milliseconds because of a failing cache node. The alarm never fired.

What should the team change?

A. Lower the static threshold to 150 milliseconds

B. Replace the static threshold with a CloudWatch anomaly detection band

C. Change datapoints to alarm from 3 of 3 to 1 of 1

D. Shorten the period from five minutes to one minute

◆ PAUSE AND DECIDE

What should the team change?

A. Lower the threshold to 150ms

Catches Sunday and turns the weekday peak into a permanent alarm. It moves the noise, it does not remove it.

B. Anomaly detection band

Learns the daily and weekly shape, so 160ms is anomalous on a Sunday night and unremarkable at 9am on a Tuesday.

C. 1 of 1 datapoints

Makes it more sensitive to spikes at any hour, which worsens the weekday false alarms and still misses 160ms overall.

D. One-minute period

A frequency change. The threshold is still a single number, so it is still wrong at one end of the day or the other.

THE SIGNAL

The same value is fine at 9am and alarming at 2am. One number cannot express that. Seasonality is the anomaly detection signal.

> An EC2 instance running a licensed legacy application cannot be replaced automatically, because its licence is bound to the instance ID and its private IP address is hard-coded in three partner firewalls.
>
> Twice this year the instance became unreachable. Both times the EC2 system status check failed while the instance status check stayed healthy, and an engineer manually stopped and started the instance to fix it.

What should the engineer configure?

A. An alarm on StatusCheckFailed_Instance with the reboot action

B. An alarm on StatusCheckFailed_System with the recover action

C. Put the instance in an Auto Scaling group of one, so a failed health check replaces it

D. An alarm on StatusCheckFailed_System with the terminate action

◆ PAUSE AND DECIDE

What should the engineer configure?

A Instance check, reboot

Watches the wrong check. The instance check was healthy both times, so this alarm would never have fired.

B System check, recover

System check failure means the host is broken. Recovery migrates to new hardware and keeps the instance ID and the private IP, which the constraints demand.

C Auto Scaling group of one

The architecturally fashionable answer, and it breaks both stated constraints. A replacement gets a new instance ID and a new private IP.

D System check, terminate

Right check, catastrophic action. It destroys the licensed instance rather than moving it.

THE SIGNAL

Two constraints in the first paragraph name the answer: keep the instance ID and keep the private IP. Recovery preserves both; replacement preserves neither.

The five things to carry forward

- **M out of N, over a window.** Not a tripwire. That explains most "why did it not fire" questions.
- **Ask what silence means.** Errors go notBreaching; heartbeats go breaching. DynamoDB defaults to ignore.
- **Actions fire on the transition**, not repeatedly. Auto Scaling actions are the exception.
- **System check means recover; instance check means reboot.** Recovery keeps the ID and the private IP.
- **Condition-based silence is a composite suppressor. Schedule-based silence is a mute rule.**

> ON THE EXAM
>
> Next: CloudWatch Logs and the unified agent, which is where the metrics you cannot get any other way come from.